

From Fitted Model to R Package

A Random Forest Case Study for Statistics Educators

A step-by-step guide for turning a trained R model into an installable, versioned, testable package

Who This Guide Is For

This article is written for intermediate and advanced R users – particularly university lecturers and researchers in statistics – who have already built a predictive model in R and want to distribute it so that colleagues, students, or collaborators can install it with a single command and use it without re-running training code or copy-pasting scripts. You should already be comfortable writing R functions, fitting models, and using RStudio. No prior package-development experience is assumed.

Because the audience for this guide builds statistical models for a living, the worked example goes one step beyond a toy linear model: it packages a **random forest classifier**, which introduces two realities every package author eventually meets – a dependency on compiled code, and a model that is trained stochastically rather than fit by a closed-form formula. Every technique shown here, however, applies without modification to simpler models (linear regression, GLMs) and to more elaborate ones (tidymodels workflows, xgboost, deep learning).

Contents

1	Why Package a Model?	2
2	Overall Workflow	2
3	Step 1: Scaffold the Package	3
4	Step 2: Train the Model and Store It as Internal Data	4
5	Step 3: Write User-Facing Functions	5
6	Step 4: Document Everything	7
7	Step 5: Write Tests	7
8	Step 6: Check, Build, and Install Locally	8
9	Step 7: Distribute the Package	9
9.1	GitHub (most common for teaching/research)	9
9.2	A private or university R-Universe / drat repository	9
9.3	CRAN	9
10	Handling Larger or More Complex Models	9
11	Quick Reference Checklist	10
12	Further Reading	10

1 Why Package a Model?

A model that lives only as an `.RData` file and a loose script is fragile: it is hard to version, hard to document, and hard for someone else to reproduce. Wrapping a model in an R package gives you, for free:

- **A stable installation path** – `install.packages()` or `devtools::install_github()` instead of “download this folder and source three files.”
- **Versioning** – the `DESCRIPTION` file tracks a version number, so users and students can pin the exact model they used in an assignment or paper.
- **Documentation that travels with the code** – `?predict_infertility_risk` works for anyone who installs the package, in every R session, with no extra files to find.
- **Reproducibility** – dependencies, the trained object, and the prediction logic are bundled together and versioned as one unit.
- **Testability** – unit tests catch silent breakage when you retrain or refactor.

Tip If you only need to share a model with one or two colleagues for a week, a package may be overkill – consider a simple script with `saveRDS()`. Build a package when the model will be reused across semesters, by multiple people, or needs a stable public interface.

2 Overall Workflow

At a high level, packaging a model involves five stages:

1. **Scaffold** the package structure.
2. **Train** the model once, then **serialize** it into the package as internal data.
3. **Wrap** the model in user-facing functions (e.g. `predict_*`()).
4. **Document and test** every exported function.
5. **Build, check, and distribute** the package.

The Running Example

The rest of this guide builds a package called `fertilityrisk`. It wraps a **random forest classifier** (fit with the `ranger` package) trained on `datasets::infert` – the classic Trichopoulos et al. case-control dataset on secondary infertility that ships with base R, so no external data download is required. The model estimates the probability that a woman is a “case” (experienced secondary infertility) from four reproductive-history variables: age, parity, and the number of prior induced and spontaneous abortions.

This example was chosen deliberately because it forces us to confront two issues that a plain `lm()` example would hide: the package now depends on compiled code (`ranger` is written in C++ via `Rcpp`), and the model is fit stochastically, which raises reproducibility questions that a linear model’s closed-form solution never does. Both issues, and their solutions, are called out as they arise.

3 Step 1: Scaffold the Package

Use the `usethis` package to generate a clean package skeleton. This avoids hand-writing boilerplate and follows CRAN-compatible conventions from the start.

```
install.packages(c("usethis", "devtools", "roxygen2", "testthat"))

library(usethis)
create_package("~/projects/fertilityrisk")
```

This creates a new RStudio project with the following structure:

```
fertilityrisk/
  DESCRIPTION
  NAMESPACE
  R/
  man/
  fertilityrisk.Rproj
```

Immediately configure a few essentials:

```
use_mit_license("Your Name")      # or use_gpl3_license(), etc.
use_roxygen_md()                  # enables roxygen2 + markdown docs
use_testthat(3)                   # sets up the testing framework
use_package("ranger")             # declares a runtime dependency
```

Open `DESCRIPTION` and fill in the metadata:

```
Package: fertilityrisk
Title: Predict Secondary Infertility Risk from Reproductive History
Version: 0.1.0
Authors@R: person("Jane", "Doe", email = "jane@university.edu",
  role = c("aut", "cre"))
Description: Provides a pretrained random forest model and helper
  functions to estimate the probability of secondary infertility
  from age, parity, and prior abortion history, based on the
  classic infert case-control dataset. Intended for teaching
  predictive modeling and R package development.
License: MIT + file LICENSE
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.1
Depends: R (>= 4.1.0)
Imports: ranger
```

Tip Use a package name that is lowercase, alphanumeric, and not already on CRAN. Check availability with `available::available("fertilityrisk")`. Note that `ranger` is listed under `Imports`, not `Depends`: your package uses it internally, but installing `fertilityrisk` should not silently attach `ranger` into the user's session.

4 Step 2: Train the Model and Store It as Internal Data

The trained model object should *not* be retrained every time someone loads your package – it should be fit once and shipped inside the package as serialized data. R packages support this cleanly via the `data/` directory (for data objects users can access directly) or `R/sysdata.rda` (for internal-only objects, invisible to the user but available to your own functions).

Create a script that trains the model. Keep this script *outside* the `R/` folder, since it is a one-time build step, not package code that runs on every load. The convention is to put it under `data-raw/`.

```
use_data_raw("infern_model")
```

This opens `data-raw/infern_model.R`. Write your training logic there:

```
## data-raw/infern_model.R
library(usethis)
library(ranger)

data(infern, package = "datasets")

# A fixed seed makes the fitted forest reproducible run-to-run.
# num.threads = 1 additionally pins down tree-building order, so the
# fitted model -- and any pinned test values built from it -- are
# identical no matter how many cores the training machine has.
set.seed(123)
infern_rf <- ranger::ranger(
  formula      = factor(case) ~ age + parity + induced + spontaneous,
  data         = infern,
  probability   = TRUE,
  num.trees    = 500,
  importance    = "impurity",
  num.threads   = 1
)

# Save as internal data, not exported to users directly
usethis::use_data(infern_rf, internal = TRUE, overwrite = TRUE)
```

Run this script once:

```
source("data-raw/infern_model.R")
```

This creates `R/sysdata.rda`, a binary file containing `infern_rf`, automatically loaded into your package's internal namespace whenever the package loads. Users never see `infern_rf` directly – they interact with it only through the functions you write in Step 3.

Common Pitfall Stochastic models need a pinned seed and a pinned thread count. A linear model fit by `lm()` always returns the same coefficients. A random forest does not: it resamples observations and candidate splits at every tree. Without `set.seed()`, retraining produces a slightly different model every time, which silently changes your package's predictions between builds. With parallelized algorithms like `ranger`, `set.seed()` alone is not always sufficient for bit-for-bit reproducibility across

machines with different core counts – setting `num.threads = 1` during the one-time data-raw/ training run removes that variability entirely. (The package’s runtime prediction function does not need this restriction, since prediction from an already-fitted forest is deterministic.)

If the trained object were large (tens of MB – less of a concern for a 500-tree forest on four predictors, but common for deep learning models or forests with thousands of trees on wide data), you would want to reconsider whether it belongs in the package at all. CRAN enforces a soft 5MB package-size limit. Large models are often better distributed via a companion data package, a release asset on GitHub, or downloaded on first use with a caching mechanism (e.g. the `pins` package).

If you instead want users to be able to inspect the fitted forest directly (e.g. for a teaching package where students examine out-of-bag error or tree structure), export it as regular package data instead of internal data:

```
usethis::use_data(infert_rf, overwrite = TRUE) # goes to data/infert_rf.
```

Exported data objects also need a documentation file, covered in Step 4.

5 Step 3: Write User-Facing Functions

Create an R script for your prediction logic:

```
use_r("predict_infertility_risk")
```

Write a clean, validated wrapper function around the internal model object:

```
#' Predict secondary infertility risk
#
# Estimates the probability that a woman is a "case" in the sense of
# the classic `infert` case-control study (i.e. experienced secondary
# infertility), using a random forest trained on age, parity, and
# prior abortion history.
#
# @param age Numeric. Age in years.
# @param parity Numeric. Number of prior pregnancies.
# @param induced Numeric. Number of prior induced abortions (0, 1, or 2+).
# @param spontaneous Numeric. Number of prior spontaneous abortions
# (0, 1, or 2+).
#
# @return A numeric vector of predicted probabilities (one per input
# row) of being a case, each in  $[0, 1]$ .
#
# @examples
# predict_infertility_risk(age = 30, parity = 2, induced = 1, spontaneous
# = 0)
#
# @export
predict_infertility_risk <- function(age, parity, induced, spontaneous) {
  inputs <- list(age = age, parity = parity,
    induced = induced, spontaneous = spontaneous)
```

```

if (!all(vapply(inputs, is.numeric, logical(1)))) {
  stop("`age`, `parity`, `induced`, and `spontaneous` must all be ",
       "numeric.", call. = FALSE)
}
if (length(unique(lengths(inputs))) > 1) {
  stop("`age`, `parity`, `induced`, and `spontaneous` must all be ",
       "the same length.", call. = FALSE)
}

newdata <- data.frame(age = age, parity = parity,
                      induced = induced, spontaneous = spontaneous)

# ranger's predict() on a probability forest returns a matrix of
# class probabilities with one column per outcome level ("0", "1").
pred <- stats::predict(infert_rf, data = newdata)
as.numeric(pred$predictions[, "1"])
}

```

A second exported function exposes model interpretation – something statisticians, unlike many ML engineers, will almost always want alongside raw predictions:

```

#' Variable importance for the infertility risk model
#'
#' Returns impurity-based variable importance scores from the
#' underlying random forest, useful for classroom discussion of
#' which reproductive-history variables drive the model's predictions.
#'
#' @return A named numeric vector of importance scores, sorted from
#' most to least important.
#'
#' @examples
#' infertility_risk_importance()
#'
#' @export
infertility_risk_importance <- function() {
  sort(infert_rf$variable.importance, decreasing = TRUE)
}

```

Note four important conventions:

- Functions reference `infert_rf` directly – it is available automatically because `R/sysdata.rda` is loaded with the package namespace.
- Use `stats::predict()` (fully qualified) rather than attaching packages with `library()` inside package code; `predict()` here dispatches to `ranger`'s method because `infert_rf` carries class `"ranger"`.
- Validate inputs and fail with informative errors (`stop(..., call. = FALSE)`) rather than letting a cryptic internal matrix-indexing error surface to the user.
- The `@export` roxygen tag makes a function visible to users; helper functions you do not want to export simply omit this tag.

Tip For larger model packages, separate concerns into multiple files: `R/predict.R` for prediction functions, `R/preprocess.R` for feature engineering helpers, `R/utils.R` for internal utilities. One exported concept per file, named to match, is the tidyverse style convention.

6 Step 4: Document Everything

R packages use roxygen2 comments (the `#'` lines above each function) to auto-generate `.Rd` help files. After writing or editing any documentation comment, regenerate the docs and the `NAMESPACE` file:

```
devtools::document()
```

This reads every `@` tag in your `R/` scripts and produces `man/predict_infertility_risk.Rd` and `man/infertility_risk` plus updates `NAMESPACE` with the correct `export()` and `import()` directives. Never hand-edit `NAMESPACE` or files in `man/` – they are generated artifacts.

If you exported the model object itself in Step 2, document it too, typically in `R/data.R`:

```
#' Pretrained infertility risk random forest
#'
#' A `ranger` random forest object predicting secondary infertility
#' case status from `age`, `parity`, `induced`, and `spontaneous`,
#' fitted on the built-in `infert` dataset.
#'
#' @format An object of class `ranger`.
#' @source `ranger::ranger(factor(case) ~ age + parity + induced +
#'   spontaneous, data = infert, probability = TRUE)`
"infert_rf"
```

Finally, write a top-level `README.Rmd` (rendered to `README.md`) with installation instructions and a minimal usage example – this is the first thing anyone sees on GitHub:

```
use_readme_rmd()
```

Tip A good `README` for a model package includes: what the model predicts, what data it was trained on, its intended use case (and explicit limitations – the `infert` data is a small, decades-old teaching dataset, not a validated clinical instrument), and a copy-pasteable usage example.

7 Step 5: Write Tests

Testing a model wrapper is different from testing arbitrary code – you are mainly checking the *interface* (input validation, output shape, type stability, and value range) rather than re-deriving the statistics from scratch.

```
use_test("predict_infertility_risk")
```

```
test_that("predict_infertility_risk returns correct length and type", {
  result <- predict_infertility_risk(age = c(30, 40), parity = c(2, 1),
                                     induced = c(1, 0), spontaneous = c(0,
```

```

1))

expect_type(result, "double")
expect_length(result, 2)
})

test_that("predict_infertility_risk returns valid probabilities", {
  result <- predict_infertility_risk(age = 30, parity = 2,
                                     induced = 1, spontaneous = 0)
  expect_true(all(result >= 0 & result <= 1))
})

test_that("predict_infertility_risk matches a known reference value", {
  # Reference value computed once (with num.threads = 1 at training
  # time) and pinned here, to catch silent model drift if infert_rf
  # is ever retrained/overwritten with different settings.
  result <- predict_infertility_risk(age = 30, parity = 2,
                                     induced = 1, spontaneous = 0)
  expect_equal(round(result, 2), 0.31, tolerance = 0.01)
})

test_that("predict_infertility_risk errors on invalid input", {
  expect_error(predict_infertility_risk(age = "thirty", parity = 2,
                                       induced = 1, spontaneous = 0))
  expect_error(predict_infertility_risk(age = c(30, 40), parity = 2,
                                       induced = 1, spontaneous = 0))
})

```

Run the full suite with:

```
devtools::test()
```

Common Pitfall Always include at least one “reference value” test that pins a known prediction to a fixed number, as above. If someone accidentally retrains and overwrites `sysdata.rda` with a different model – or forgets to set a seed – this test catches the silent behavior change immediately. For stochastic models, this test is doing double duty: it verifies both your wrapper code *and* the reproducibility of the training pipeline that produced `sysdata.rda` in the first place.

8 Step 6: Check, Build, and Install Locally

Before sharing the package, run R’s built-in validator. This checks documentation completeness, namespace correctness, examples that actually run, declared-but-unused dependencies, and dozens of other conventions:

```
devtools::check()
```

Resolve every ERROR and WARNING; NOTES are often acceptable but worth reading. A clean `check()` is required if you ever intend to submit to CRAN, and is good discipline even for internal packages. Because `fertilityrisk` depends on `ranger`, a compiled-code package, `check()` will also confirm that the compiled dependency installs and loads correctly – a class of problem that never arises with pure-stats models.

Then install and test it as a user would:


```
devtools::install()
library(fertilityrisk)
predict_infertility_risk(age = 35, parity = 1, induced = 0, spontaneous =
  1)
infertility_risk_importance()
```

9 Step 7: Distribute the Package

You have three common distribution routes, in increasing order of formality:

9.1 GitHub (most common for teaching/research)

Push the package to a GitHub repository. Students and colleagues install it with:

```
# install.packages("remotes")
remotes::install_github("yourusername/fertilityrisk")
```

Tag releases (v0.1.0, v0.2.0, ...) so users can pin a specific version:

```
remotes::install_github("yourusername/fertilityrisk@v0.1.0")
```

9.2 A private or university R-University / drat repository

If you publish multiple teaching packages, an [R-universe](#) repository lets users install with plain `install.packages()` pointed at a custom repo URL – no remotes needed and no GitHub token required for private forks.

9.3 CRAN

For general-purpose, broadly useful packages, CRAN submission via `devtools::release()` is the gold standard, but requires strict compliance (portable code, no writing outside `tempdir()`, fast examples, compiled dependencies that build cleanly on Windows/macOS/Linux, etc.). Most single-course teaching packages are well served by GitHub distribution instead.

Tip For a course, consider adding installation instructions directly to your syllabus or LMS page: `remotes::install_github("yourusername/fertilityrisk")`. Pin a tag per semester so a later model update never silently changes a past assignment's expected output.

10 Handling Larger or More Complex Models

The random-forest example above already covers two of the trickiest real-world issues – compiled-code dependencies and stochastic training – but a few further situations are worth knowing about:

- **tidymodels workflows / caret models / xgboost:** store the fitted workflow, train, or `xgb.Booster` object the same way via `usethis::use_data(..., internal = TRUE)`. For `xgboost` specifically, prefer `xgb.save()` to a file bundled under `inst/extdata/` and load it lazily in `.onLoad()`, since `xgb.Booster` objects do not always serialize reliably via plain `.rda`.
- **Deep learning models (torch, keras):** these carry external-language state that does not survive stan-

dard R serialization. Save native checkpoints under `inst/extdata/` and reconstruct the model object at load time, typically via a package-level cache and a `.onLoad()` hook.

- **Mixed-effects and Bayesian models (lme4, brms, rstan):** these often bundle a compiled sampler or a large posterior draw matrix. The same `data-raw/ + sysdata.rda` pattern works, but pay close attention to object size – a saved `brmsfit` can easily exceed CRAN’s soft size limit and may be better served by exporting a summarized posterior instead of the full fit.
- **Models with heavy dependencies:** list them under `Imports` in `DESCRIPTION`, not `Depends`, so they load only when actually needed and do not clutter the user’s namespace.
- **Models that need periodic retraining:** keep the `data-raw/` training script under version control and re-run Step 2 whenever new data arrives, bumping the package `Version` each time so downstream users can detect the change.

11 Quick Reference Checklist

- ☐ `create_package()` scaffolds the project
- ☐ License and `DESCRIPTION` metadata filled in; model dependencies (e.g. `ranger`) listed under `Imports`
- ☐ Model trained once in `data-raw/` with a fixed seed (and, for parallel algorithms, a fixed thread count), saved via `use_data(internal = TRUE)`
- ☐ Prediction function(s) written with input validation and `@export`
- ☐ `devtools::document()` run after every roxygen edit
- ☐ README with install instructions and a working example
- ☐ Tests cover shape, type, valid output range, invalid input, and a pinned reference value
- ☐ `devtools::check()` passes with no errors or warnings
- ☐ Package pushed to GitHub with a tagged release

12 Further Reading

- Wickham, H. & Bryan, J. *R Packages*, 2nd edition – <https://r-pkgs.org>
- `usethis` package documentation – <https://usethis.r-lib.org>
- `roxygen2` documentation – <https://roxygen2.r-lib.org>
- `ranger` package documentation – <https://cran.r-project.org/package=ranger>
- R-universe, for lightweight package repositories – <https://r-universe.dev>

This guide uses a random forest classifier for concreteness, since it surfaces the compiled-code and reproducibility questions that simpler models hide. The same six-step workflow – scaffold, serialize, wrap, document, test, distribute – applies unchanged to simpler models (`lm`, `glm`) and to more elaborate ones (tidymodels workflows, `xgboost`, mixed-effects and Bayesian models, or `torch`).